



Transport Layer Quick Revision Notes

1. Introduction & Services

Transport Layer Purpose: Provides logical communication between application processes running on different hosts

Key Responsibilities

- **Process-to-Process Communication:** Extends host-to-host to process-to-process delivery
- **Segmentation:** Breaks messages into segments with transport header
- **Multiplexing/Demultiplexing:** Routes data to correct application

TCP vs UDP Comparison

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (retransmission, ordering)	Unreliable (best effort)
Flow Control	Yes	No
Congestion Control	Yes	No
Header Size	20 bytes (minimum)	8 bytes
Speed	Slower	Faster
Use Cases	HTTP, FTP, Email	DNS, Streaming, Gaming

2. Multiplexing & Demultiplexing

Port Numbers

- **Well-known ports:** 0-1023 (HTTP=80, FTP=21, DNS=53)
- **Ephemeral ports:** 1024-65535 (dynamically assigned to clients)
- **16-bit field:** Range 0-65535

UDP Multiplexing

UDP Socket Identification: 2-tuple (Destination IP, Destination Port)

Behavior: All segments with same destination IP+Port go to SAME socket

TCP Multiplexing

TCP Socket Identification: 4-tuple (Source IP, Source Port, Dest IP, Dest Port)

Behavior: Each unique 4-tuple gets DIFFERENT socket

KEY DIFFERENCE: TCP creates separate sockets for each connection; UDP uses one socket for all clients

3. UDP Protocol

UDP Segment Structure (8 bytes header)

- Source Port (16 bits)
- Destination Port (16 bits)
- Length (16 bits) - includes header + data
- Checksum (16 bits)

UDP Checksum

Calculation:

1. Sum all 16-bit words (with wraparound)
2. Take 1's complement of sum
3. Result = checksum

Verification:

Add all words including checksum

If result = all 1s → No error detected

If any 0 → Error detected

UDP provides ERROR DETECTION but NOT ERROR RECOVERY (corrupted segments are discarded)

4. Reliable Data Transfer (RDT) Principles

RDT 1.0 - Perfect Channel

- No bit errors, no packet loss
- Simple: sender sends, receiver receives

RDT 2.0 - Channel with Bit Errors

- **ARQ Mechanisms:**
 - Error detection (checksum)
 - Feedback (ACK/NAK)
 - Retransmission
- **Problem:** What if ACK/NAK corrupted?

RDT 2.1 - Handling Corrupted ACK/NAK

- **Solution:** Add sequence numbers (0, 1)
- Receiver discards duplicate packets
- Stop-and-wait protocol

RDT 2.2 - NAK-Free Protocol

- Uses only ACKs with sequence numbers
- Duplicate ACK = implicit NAK

RDT 3.0 - Channel with Loss

- **New mechanism:** Timeout timer
- Sender retransmits if no ACK before timeout
- Handles both loss and corruption

STOP-AND-WAIT PROBLEM: Very inefficient due to waiting for RTT
Utilization = $(L/R) / (RTT + L/R) \approx 0.027\%$ for typical values

5. Pipelined Protocols

Pipelining Benefits

Utilization with Pipelining:

$$U = (N \times L/R) / (RTT + L/R)$$

Where N = number of packets in pipeline

Go-Back-N (GBN)

- **Window size:** N (sender can have N unACKed packets)
- **Cumulative ACKs:** $\text{ACK}(n) = \text{"received all up to } n\text{"}$
- **Receiver:** Only accepts in-order packets, discards out-of-order
- **On timeout/NAK:** Retransmit ALL packets from base to $\text{nextSeqNum}-1$
- **Timer:** Single timer for oldest unACKed packet

Selective Repeat (SR)

- **Window size:** N (both sender and receiver)
- **Individual ACKs:** Each packet ACKed separately
- **Receiver:** Buffers out-of-order packets
- **On timeout:** Retransmit ONLY that packet
- **Timer:** Individual timer for each packet

SR Dilemma: Window size $\leq$ Sequence space / 2

Otherwise receiver can't distinguish new packets from retransmissions

6. TCP Segment Structure

TCP Header Fields (20 bytes minimum)

- **Source/Dest Port:** 16 bits each
- **Sequence Number:** 32 bits (byte stream number)
- **ACK Number:** 32 bits (next expected byte)
- **Header Length:** 4 bits
- **Flags:** URG, ACK, PSH, RST, SYN, FIN
- **Receive Window:** 16 bits (for flow control)
- **Checksum:** 16 bits
- **Options:** Variable length

Important TCP Concepts

Sequence Numbers: Count BYTES, not segments
First byte of data in segment = sequence number

ACK Numbers: Next expected byte (cumulative)
 $\text{ACK}(n) = \text{"received all bytes up to } n-1 \text{"}$

MSS (Maximum Segment Size):
 $\text{MSS} = \text{MTU} - \text{IP header} - \text{TCP header}$
Typical: $1500 - 20 - 20 = 1460$ bytes

7. TCP RTT & Timeout

EstimatedRTT (Exponentially Weighted Moving Average):
$$\text{EstimatedRTT} = (1-\alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}$$

 $\alpha = 0.125$ (recommended)

DevRTT (RTT Variation):

$$\text{DevRTT} = (1-\beta) \times \text{DevRTT} + \beta \times |\text{SampleRTT} - \text{EstimatedRTT}|$$

$\beta = 0.25$ (recommended)

TimeoutInterval:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \times \text{DevRTT}$$

Timeout Handling

- **On timeout:** Double TimeoutInterval (exponential backoff)
- **On new ACK:** Recalculate using formulas
- **Fast Retransmit:** 3 duplicate ACKs → retransmit immediately

8. TCP Reliable Data Transfer

Key Mechanisms

- **Cumulative ACKs:** Like GBN
- **Out-of-order buffering:** Like SR
- **Single retransmission timer**
- **Fast Retransmit:** 3 dup ACKs = retransmit without waiting for timeout

TCP is a HYBRID: GBN-style ACKs + SR-style buffering

9. Flow Control

Purpose: Prevent sender from overwhelming receiver's buffer

Mechanism: Receiver advertises RcvWindow in every segment

$$\text{RcvWindow} = \text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$$

Sender constraint:

$\text{LastByteSent} - \text{LastByteAcked} \leq \text{RcvWindow}$

If $\text{RcvWindow} = 0$, sender still sends 1-byte segments to probe for window updates

10. TCP Connection Management

3-Way Handshake (Connection Establishment)

1. **SYN:** Client → Server (SYN=1, seq=x)
2. **SYN-ACK:** Server → Client (SYN=1, ACK=1, seq=y, ack=x+1)
3. **ACK:** Client → Server (ACK=1, seq=x+1, ack=y+1)

4-Way Handshake (Connection Termination)

1. **FIN:** Client → Server (FIN=1)
2. **ACK:** Server → Client (ACK=1)
3. **FIN:** Server → Client (FIN=1)
4. **ACK:** Client → Server (ACK=1)

Why 3-way not 2-way? To handle delayed duplicate SYN packets and synchronize both sides' sequence numbers

11. Congestion Control

Congestion Causes & Costs

- **Scenario 1 (Infinite buffers):** Large queuing delays
- **Scenario 2 (Finite buffers):** Unnecessary retransmissions waste bandwidth
- **Scenario 3 (Multi-hop):** Upstream work wasted when downstream drops packets

Approaches

- **End-to-End:** No explicit feedback from network (TCP)
- **Network-Assisted:** Routers provide feedback (ECN)

12. TCP Congestion Control (Classic)

Congestion Window (cwnd): Sender's limit on unACKed data
 $\text{LastByteSent} - \text{LastByteAcked} \leq \min(\text{cwnd}, \text{RcvWindow})$

Sending Rate $\approx \text{cwnd} / \text{RTT}$

Three States

1. Slow Start

- **Initial cwnd:** 1 MSS
- **Growth:** Double every RTT (exponential)
- **Implementation:** $\text{cwnd} += 1 \text{ MSS}$ for each ACK received
- **Exit conditions:**
 - Timeout $\rightarrow$ set $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1$, restart Slow Start
 - $\text{cwnd} \geq \text{ssthresh} \rightarrow$ enter Congestion Avoidance
 - 3 dup ACKs $\rightarrow$ set $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = \text{ssthresh} + 3$, enter Fast Recovery

2. Congestion Avoidance

- **Growth:** Linear (additive increase)
- **Implementation:** $\text{cwnd} += 1 \text{ MSS}$ per RTT
- **Exit conditions:**
 - Timeout $\rightarrow$ set $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1$, go to Slow Start
 - 3 dup ACKs $\rightarrow$ set $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = \text{ssthresh} + 3$, go to Fast Recovery

3. Fast Recovery

- **Entry:** After 3 duplicate ACKs
- **Action:** Retransmit missing segment
- **cwnd:** Inflate by 1 MSS for each additional dup ACK
- **Exit:** When new ACK arrives $\rightarrow \text{cwnd} = \text{ssthresh}$, go to Congestion Avoidance

AIMD: Additive Increase, Multiplicative Decrease

Increase: +1 MSS per RTT

Decrease: $\text{cwnd} = \text{cwnd} / 2$ on loss

TCP Variants

- **TCP Tahoe:** No Fast Recovery (always slow start after loss)
- **TCP Reno:** Has Fast Recovery
- **TCP CUBIC:** Cubic function for cwnd growth (modern default)

13. Advanced Congestion Control

Explicit Congestion Notification (ECN)

- **Network-assisted approach**
- Router marks IP packet header when congested
- Receiver echoes ECN to sender in TCP header
- Sender reduces cwnd without waiting for loss

Delay-Based Congestion Control

- **Idea:** Detect congestion before loss occurs
- Monitor RTT increases (indicates queue buildup)
- **Example:** TCP Vegas
- Measure throughput vs expected throughput

14. TCP Fairness

Fairness Goal:

K TCP connections sharing bottleneck link of capacity R
Each should get R/K throughput on average

AIMD achieves fairness (converges to equal sharing)

Causes of Unfairness

- **Parallel connections:** Application opens multiple TCP connections
- **UDP traffic:** No congestion control, can crowd out TCP

15. Important Formulas Summary

Utilization (Stop-and-Wait):

$$U = (L/R) / (RTT + L/R)$$

Throughput:

$$\text{Throughput} = U \times R$$

TCP Throughput (Reno):

$$\text{Average Throughput} \approx (1.22 \times \text{MSS}) / (RTT \times \sqrt{L})$$

Where L = loss probability

MSS Calculation:

$$\begin{aligned} \text{MSS} &= \text{MTU} - \text{IP_header} - \text{TCP_header} \\ &= 1500 - 20 - 20 = 1460 \text{ bytes (typical)} \end{aligned}$$

Flow Control:

$$\text{RcvWindow} = \text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$$

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{RcvWindow}$$

Congestion Control:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min(\text{cwnd}, \text{RcvWindow})$$

$$\text{Sending Rate} \approx \text{cwnd} / \text{RTT}$$

16. Key Differences Quick Reference

Aspect	GBN	SR	TCP
ACKs	Cumulative	Individual	Cumulative
Out-of-order	Discard	Buffer	Buffer
Retransmit	All from base	Only lost	Only lost (fast retransmit)
Timer	Single	Per packet	Single
Window	Sender only	Both sides	Both sides

17. Common Exam Traps

- TCP sequence numbers count BYTES, not segments
- ACK number = next expected byte (not last received)
- UDP has NO connection setup, NO flow control, NO congestion control
- Fast retransmit needs 3 DUPLICATE ACKs (4 total ACKs for same data)
- Slow start is EXPONENTIAL, Congestion Avoidance is LINEAR
- On timeout: $cwnd = 1$, $ssthresh = cwnd/2$ (ALWAYS go to Slow Start)
- On 3 dup ACKs: Fast Recovery ($cwnd = ssthresh + 3$, stay in Fast Recovery)
- SR window size must be $\leq \text{sequence space} / 2$
- Flow control = receiver buffer management
- Congestion control = network capacity management